PYTHON BASICS – COMPILATION, INTERPRETATION & PYTHON VERSIONS

1 COMPILATION

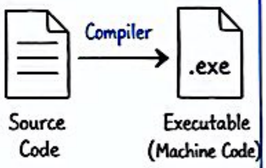
Definition

Compilation translates the entire source code into machine code before the program runs.

Output :

- Executable file (e.g. .exe in Windows)

Done by : Compiler (Translator)



Advantages

- ✓ Faster program execution
- ✓ End user does NOT need the compiler
- ✓ Machine code is difficult to read → protects source code

Disadvantages

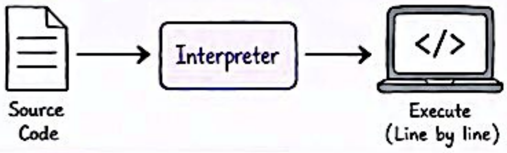
- ✗ Compilation takes time after every code modification
- ✗ Different hardware/platforms require different compilers

2 INTERPRETATION

Definition

Interpretation translates and executes the program line by line every time it runs.

Done by : Interpreter



Advantages

- ✓ Run the program immediately after writing/editing
- ✓ Same source code can run on different operating systems (with the correct interpreter)
- ✓ No need to compile separately for each platform

Disadvantages

- ✗ Slower execution because the interpreter translates while running
- ✗ Both the programmer and the end user need the interpreter

3 COMPILER vs INTERPRETER

Feature	Compiler	Interpreter
Translation	Translates once	Translates every run
Output	Produces executable (.exe)	No executable produced
Execution Speed	Faster execution	Slower execution
End User Requirement	End user doesn't need compiler	End user needs interpreter
Code Protection	Better – source code is not visible (machine code)	Lower – source code remains visible
Platform	Platform dependent	More portable (with interpreter)

Remember!

- ✓ Compiler → Translate Once, Run Many Times
- ✓ Interpreter → Translate Every Time You Run

4 PYTHON 2 vs PYTHON 3

Python 2

- Older version
- Only receives bug fixes & security updates
- No major new features
- Still used for old applications

VS

Python 3

- Current and recommended version
- Actively developed
- Includes new features and improvements
- Best choice for all new projects

5 COMPATIBILITY

Python 2 ✗ Python 3

- Python 2 programs cannot run directly in Python 3.
- Python 3 programs cannot run in Python 2.
- Old Python 2 code usually needs modification before running in Python 3.

Python 3 Versions

✓ Newer Python 3 versions are backward compatible with older Python 3 versions.

Example: Python 3.10 ✓ → Python 3.11 ✓ → Python 3.12 ✓

Most code works!

6 KEY TERMS TO REMEMBER

Compiler

- Translates source code once
- Produces machine code
- Creates executable file
- Faster execution

Interpreter

- Translates every execution
- Executes line by line
- Slower
- Needs interpreter installed

Python 2 vs Python 3

- Not compatible
- Python 2 = old
- Python 3 = current and improved

Python 3 Versions

- Newer versions are backward compatible
- Code written for older Python 3 usually works on newer Python 3

☆ **NOTE:** If you're going to start a new Python project, you should use Python 3, and this is the version of Python that will be used during this course.

Python Implementations You Should Know (Beginner Notes)

1 Cython

Cython

What is it?

Cython is a superset of Python that compiles Python-like code into C code and then into machine code.

Key Points for Beginners

- Not a Python interpreter.
- You write Python code with optional type annotations.
- Cython compiles it to C, giving much better performance—close to C speed.
- Useful for performance-critical parts of your program.

Use Cases / Good For

- Speeding up slow Python code
- Scientific computing (NumPy, SciPy, etc.)
- Writing C extensions easily

2 Jython

Java

What is it?

Jython is an implementation of Python written in Java. It runs on the Java Virtual Machine (JVM).

Key Points for Beginners

- Python code runs on the JVM.
- Can use Java libraries directly from Python.
- Slower than CPython for pure Python code.
- Good integration with Java world.

Use Cases / Good For

- Using Python in Java applications
- Enterprise applications
- Working with Java libraries (e.g., JDBC, Hibernate)

3 PyPy

PyPy

What is it?

PyPy is a fast, compliant implementation of Python with a Just-In-Time (JIT) compiler.

Key Points for Beginners

- Written in RPython (a restricted subset of Python).
- Uses JIT to optimize and speed up your code at runtime.
- Great for long-running Python programs.
- Not all C extensions are supported.

Use Cases / Good For

- Making Python programs run faster (especially loops)
- Data processing, simulations
- Long-running applications

4 MicroPython

MicroPython

What is it?

MicroPython is a lean and efficient implementation of Python 3, designed to run on microcontrollers and embedded systems.

Key Points for Beginners

- Runs on small devices with limited memory and power.
- Compatible with most Python 3 syntax (some modules are missing).
- Perfect for IoT and embedded projects.
- Very small size.

Use Cases / Good For

- Microcontrollers (ESP32, ESP8266, Raspberry Pi Pico, etc.)
- IoT devices and sensors
- Low-power, small memory environments

Quick Comparison

Implementation	Written In	Runs On	Speed	C Extensions Support	Best For
CPython (default)	C	Python Virtual Machine (PVM)	Normal	Yes	General purpose, most Python users
Cython	C (compiles Python to C)	Same as CPython	Very Fast (near C speed)	Yes	Speed up Python code, scientific computing
Jython	Java	Java Virtual Machine (JVM)	Slower than CPython	Limited	Java integration, enterprise applications
PyPy	RPython	PyPy VM with JIT	Fast (often faster than CPython)	Limited	Long-running programs, performance boost
MicroPython	C	Microcontrollers / Embedded systems	Fast for small tasks	No	IoT, embedded, low-power devices

Key Takeaways

- CPython is the standard Python you install from python.org.
- Cython → makes Python code faster by compiling to C.
- Jython → runs Python on the JVM and connects with Java.
- PyPy → uses JIT to make Python programs run faster.
- MicroPython → brings Python to small devices and IoT.

Tip: For most beginners → start with CPython.
Explore others when you need more speed, Java integration, or embedded support.

Cisco Networking Academy

Python Essentials 1

Course Outline

Resources

Search course outline

100%

1.0. Welcome to Python Essentials 1 4 / 4

1.1. Section 1 – Introduction to Programming 7 / 7

1.2. Section 2 – Introduction to Python 9 / 9

1.3. Section 3 – Downloading and Installing Python 5 / 5

1.4. Module 1 Completion – Module Test

PE1: Module 2. Python Data Types, Variables, Operators, and Basic I/O Operations

PE1: Module 3. Boolean Values, Conditional Execution, Loops, Lists and List Processing, Logical and Bitwise Operations

1.4. Module 1 Completion – Module Test

EN

Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8 Q9 Q10 Result Page

100%

You have scored **100%**.

Congratulations, you have passed the exam.

Reset

Review Assessment